先进计算与数字工程研究中心

PATTERN RECOGNITION AND MACHINE LEARNING

Lecture 15 – Deep Learning I: Neural Networks and Backpropagation

Chizhi Chris ZHANG

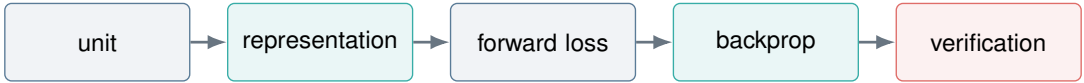
zhangchizhi@ciomp.ac.cn

Advanced Computing and Digital Technology Research Center

University of Chinese Academy of Sciences

Fall 2026

Build the model and check its gradients



A lower loss is trustworthy only after shapes, gradients, and data boundaries pass inspection.

Guiding question

How do representation, loss, and chain rule cooperate to make a network learn?

- hidden layers create nonlinear features
- output layers encode a predictive distribution
- backpropagation reuses local derivatives
- initialization and checks make learning diagnosable

From one neuron to a working two-layer network

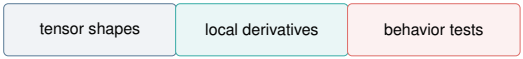
By the end

You can explain the perceptron limit, choose activations and output losses, derive vectorized backpropagation, initialize weights, and perform a numerical gradient check.

Period	Core content	Decision
1	units, perceptrons, and MLP representations	what can the architecture express?
2	forward propagation and losses	what distribution is predicted?
3	backpropagation and debugging	are gradients and training behavior correct?

Shapes, values, and derivatives are one specification

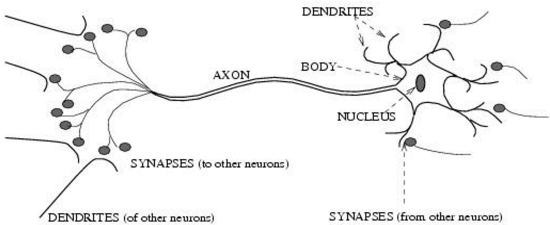
先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER



What to calculate

Every layer records input and output shapes, parameter count, activation range, loss reduction, gradient norm, numerical check, and validation behavior.

Which aspects of a biological neuron survive in a machine-learning unit?



Question

Which aspects of a biological neuron survive in a machine-learning unit?

Example

mapping dendrites, synapses, soma, and axon to a feed-forward unit

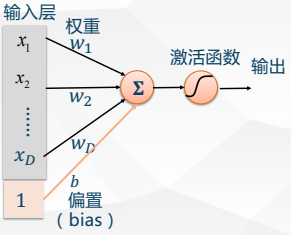
Main idea

Inputs, synaptic strengths, aggregation, and response are retained; timing, morphology, and biochemical dynamics are mostly omitted.

An artificial neuron computes a weighted sum

人工神经元（M-P模型）

■ $a = f(\boldsymbol{w}, \boldsymbol{x}) = \sigma(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x})$



■ 输入$\boldsymbol{x}$

■ 权重&偏置

$$z = \sum_{j=1}^{D} w_j x_j + b = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}$$

■ 激活函数σ ：引入非线性

■ 输出$a = \sigma(z)$

单个神经元 = 感知器，可实现线性分类

Biological inspiration and computational abstraction: model and variables

Model and assumptions

An artificial neuron forms a weighted sum, adds a bias, and applies a nonlinear activation; it is an engineering abstraction, not a biological simulation.

$$z = \sum_{j=1}^{d} w_j x_j + b, \qquad a = \phi(z)$$

Symbol	Meaning	Status
x_j	input signal	measured feature or previous activation
w_j	connection weight	learned parameter
ϕ	activation	nonlinear response

Biological inspiration and computational abstraction: worked example

Quantity	Value	Meaning
inputs	$(2, -1)$	observed
weights	$(0.5, 1)$	learned
ReLU output	$\max(0, 0) = 0$	after bias zero

What the result means

Use the analogy to motivate information flow, then reason from the mathematical unit.

Biological inspiration and computational abstraction: what to check

Main idea

Inputs, synaptic strengths, aggregation, and response are retained; timing, morphology, and biochemical dynamics are mostly omitted.

What to check

dimension consistency and activation range

Limit

taking the biological analogy literally can suggest unsupported claims about interpretability or cognition

Biological inspiration and computational abstraction: implementation

1. identify the input vector
2. state parameter dimensions
3. compute the pre-activation
4. apply the activation
5. separate analogy from model claim

Check the result

Check dimension consistency and activation range on training and held-out examples.

Common mistake

taking the biological analogy literally can suggest unsupported claims about interpretability or cognition

Biological inspiration and computational abstraction: common mistakes

Example

mapping dendrites, synapses, soma, and axon to a feed-forward unit

Explain the calculation and check dimension consistency and activation range, using a small example.

Common mistake

taking the biological analogy literally can suggest unsupported claims about interpretability or cognition

What to do

Use the analogy to motivate information flow, then reason from the mathematical unit.

Biological inspiration and computational abstraction: check your understanding

Question

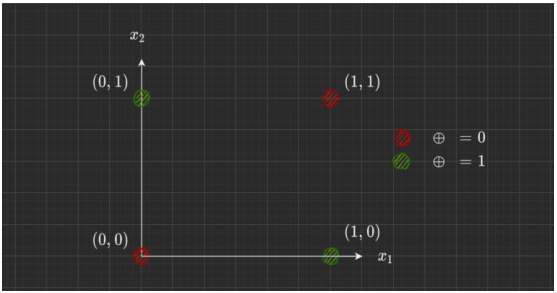
Which aspects of a biological neuron survive in a machine-learning unit?

Answer

Inputs, synaptic strengths, aggregation, and response are retained; timing, morphology, and biochemical dynamics are mostly omitted.

Decision. Use the analogy to motivate information flow, then reason from the mathematical unit.

What can one threshold unit represent, and why does XOR expose the limit?



Question

What can one threshold unit represent, and why does XOR expose the limit?

Example

classifying the four binary XOR inputs

Main idea

XOR is not linearly separable; no single line can place its opposite corners on the same side while separating the others.

Two hidden features can solve XOR

- Let $s = x_1 + x_2$, $h_1 = \mathbf{1}[s > 0.5]$, and $h_2 = \mathbf{1}[s > 1.5]$.
- For sums 0, 1, and 2, hidden patterns are $(0, 0)$, $(1, 0)$, and $(1, 1)$.
- Output $y = h_1 - h_2$ equals 1 only when the sum is 1.
- Threshold units show expressiveness; differentiable activations are used for gradient training.



Perceptron geometry and the XOR limit: model and variables

Model and assumptions

A perceptron assigns a class by the sign of an affine score, so its decision boundary is one hyperplane.

$$\widehat{y} = \mathbf{1}[w^\top x + b \geq 0]$$

Symbol	Meaning	Status
w	normal vector	boundary orientation
b	intercept	boundary displacement
$w^\top x + b = 0$	hyperplane	single linear separator

Perceptron geometry and the XOR limit: worked example

Quantity	Value	Meaning
XOR positives	$(0,1),(1,0)$	opposite corners
XOR negatives	$(0,0),(1,1)$	opposite corners
one line	impossible	needs transformed representation

What the result means

Change the representation or architecture when optimization succeeds but expressive capacity is insufficient.

The choice rule for Perceptron geometry and the XOR limit

Main idea

XOR is not linearly separable; no single line can place its opposite corners on the same side while separating the others.

What to check

linear separability and classification error

Limit

adding more training iterations cannot overcome a representation class that cannot express the target boundary

Perceptron geometry and the XOR limit: implementation

1. plot labeled points
2. write the affine inequalities
3. test simultaneous feasibility
4. identify the missing representation
5. add hidden units

Check the result

Check linear separability and classification error on training and held-out examples.

Common mistake

adding more training iterations cannot overcome a representation class that cannot express the target boundary

Perceptron geometry and the XOR limit: common mistakes

Example

classifying the four binary XOR inputs

Explain the calculation and check linear separability and classification error, using a small example.

Common mistake

adding more training iterations cannot overcome a representation class that cannot express the target boundary

What to do

Change the representation or architecture when optimization succeeds but expressive capacity is insufficient.

Perceptron geometry and the XOR limit: check your understanding

Question

What can one threshold unit represent, and why does XOR expose the limit?

Answer

XOR is not linearly separable; no single line can place its opposite corners on the same side while separating the others.

Decision. Change the representation or architecture when optimization succeeds but expressive capacity is insufficient.

How does a hidden layer turn an inseparable input problem into a separable representation?



Depth reorganizes the input through successive coordinate systems; nonlinearities prevent the composition from collapsing to one affine map.

Question

How does a hidden layer turn an inseparable input problem into a separable representation?

Example

constructing a two-hidden-unit network for XOR

Main idea

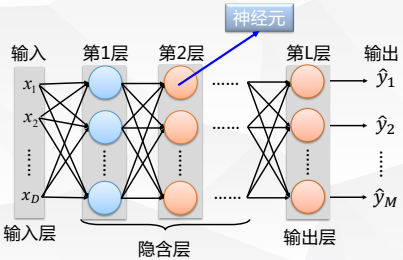
Depth reorganizes the input through successive coordinate systems; nonlinearities prevent the composition from collapsing to one affine map.

Nonlinear hidden layers learn new features

ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

全连接前馈神经网络

■深度神经网络（Deep Neural Network，DNN）：多个隐含层



Multilayer perceptrons and learned features: model and variables

Model and assumptions

An MLP composes affine maps and nonlinear activations so hidden units define reusable intermediate features.

$$a^{(\ell)} = \phi\left(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}\right)$$

Symbol	Meaning	Status
$a^{(\ell)}$	layer activation	learned representation
$W^{(\ell)}$	weight matrix	cross-unit connections
ℓ	layer index	composition depth

Multilayer perceptrons and learned features: worked example

Quantity	Value	Meaning
hidden unit 1	$x_1 + x_2 > 0.5$	OR-like feature
hidden unit 2	$x_1 + x_2 > 1.5$	AND-like feature
output	difference	XOR representation

What the result means

Use nonlinear hidden units and the smallest width that demonstrates the needed representation.

Multilayer perceptrons and learned features: checking the result

Main idea

Depth reorganizes the input through successive coordinate systems; nonlinearities prevent the composition from collapsing to one affine map.

What to check

training fit, validation error, and hidden activation diversity

Limit

stacking affine layers without nonlinearities is still only one affine transformation

What to record when running Multilayer perceptrons and learned features

1. choose hidden width
2. write every matrix shape
3. insert nonlinear activations
4. visualize hidden outputs
5. test the smallest architecture first

Check the result

Check training fit, validation error, and hidden activation diversity on training and held-out examples.

Common mistake

stacking affine layers without nonlinearities is still only one affine transformation

The warning sign for Multilayer perceptrons and learned features

Example

constructing a two-hidden-unit network for XOR

Explain the calculation and check training fit, validation error, and hidden activation diversity, using a small example.

Common mistake

stacking affine layers without nonlinearities is still only one affine transformation

What to do

Use nonlinear hidden units and the smallest width that demonstrates the needed representation.

Explain Multilayer perceptrons and learned features without naming the algorithm

Question

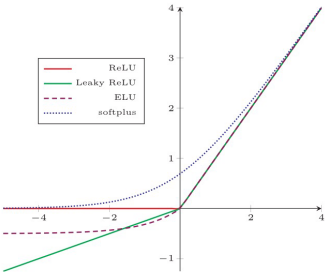
How does a hidden layer turn an inseparable input problem into a separable representation?

Answer

Depth reorganizes the input through successive coordinate systems; nonlinearities prevent the composition from collapsing to one affine map.

Decision. Use nonlinear hidden units and the smallest width that demonstrates the needed representation.

Which activation preserves useful signal and gradient for the layer's role?



Question

Which activation preserves useful signal and gradient for the layer's role?

Example

selecting hidden activations for a small tabular MLP

Main idea

ReLU avoids positive-side saturation, sigmoid maps to probability-like ranges, tanh centers outputs, and SiLU is smooth but costlier.

Compare activation functions

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

激活函数

ReLU激活函数的变种

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma x & \text{if } x \le 0 \end{cases}$$

带泄露的 ReLU

$$= \max(0, x) + \gamma \min(0, x)$$

$$\text{PReLU}_i(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma_i x & \text{if } x \le 0 \end{cases}$$

带参数的 ReLU

$$= \max(0, x) + \gamma_i \min(0, x)$$

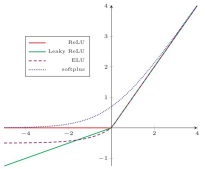
$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \gamma(\exp(x) - 1) & \text{if } x \le 0 \end{cases}$$

指数线性单元

$$= \max(0, x) + \min(0, \gamma(\exp(x) - 1))$$

$$\text{softplus}(x) = \log(1 + \exp(x))$$

Rectifier 函数的平滑版本



死亡ReLU问题

How Activation functions and derivative flow changes the representation

Model and assumptions

An activation shapes the range, sparsity, smoothness, and local derivative through which learning signals propagate.

$$\mathrm{ReLU}(z) = \max(0, z), \quad \sigma'(z) = \sigma(z)[1 - \sigma(z)]$$

Symbol	Meaning	Status
z	pre-activation	input to nonlinearity
$\phi'(z)$	local derivative	gradient gate
saturation	small derivative region	slows upstream learning

Check the direction of change in Activation functions and derivative flow

Quantity	Value	Meaning
sigmoid at 0	0.5	derivative 0.25
sigmoid at 8	0.9997	derivative near zero
ReLU at 2	2	derivative one

What the result means

Choose the activation jointly with initialization and normalization, then verify actual signal distributions.

When Activation functions and derivative flow earns a place in the pipeline

Main idea

ReLU avoids positive-side saturation, sigmoid maps to probability-like ranges, tanh centers outputs, and SiLU is smooth but costlier.

What to check

activation and gradient distributions by layer

Limit

large pre-activations can saturate sigmoid or tanh; persistent negative inputs can kill ReLU units

Activation functions and derivative flow: implementation

```
def sigmoid(z):
    z = np.clip(z, -40, 40)
    return 1.0 / (1.0 + np.exp(-z))
def relu(z):
    return np.maximum(z, 0.0)
print('Activation check:', sigmoid(np.array([-2.0, 0.0, 2.0])), relu(np.array([-2.0, 0.0, 2.0])))
```

Evidence and failure check

Check activation and gradient distributions by layer with the fitted model and data split; reject the run if large pre-activations can saturate sigmoid or tanh; persistent negative inputs can kill ReLU units.

When the objective of Activation functions and derivative flow is not enough

Example

selecting hidden activations for a small tabular MLP

Explain the calculation and check activation and gradient distributions by layer, using a small example.

Common mistake

large pre-activations can saturate sigmoid or tanh; persistent negative inputs can kill ReLU units

What to do

Choose the activation jointly with initialization and normalization, then verify actual signal distributions.

A final decision on Activation functions and derivative flow

Question

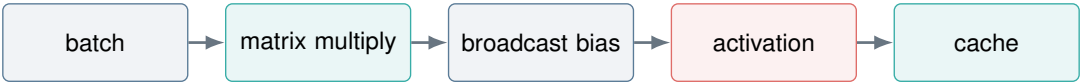
Which activation preserves useful signal and gradient for the layer's role?

Answer

ReLU avoids positive-side saturation, sigmoid maps to probability-like ranges, tanh centers outputs, and SiLU is smooth but costlier.

Decision. Choose the activation jointly with initialization and normalization, then verify actual signal distributions.

Check the shape of every forward-pass operation



Shape annotations expose most implementation errors before optimization begins.

Question

How do we make every forward-pass operation dimensionally auditable?

Example

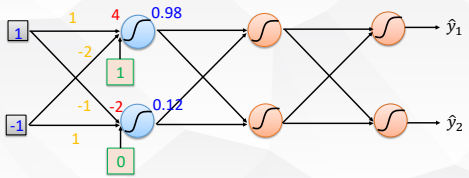
implementing a two-layer network with batch-first arrays

Main idea

Shape annotations expose most implementation errors before optimization begins.

Forward propagation is a sequence of matrix operations

矩阵操作



$$\sigma\left(\begin{bmatrix}1 & -2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\ -1\end{bmatrix}+\begin{bmatrix}1\\ 0\end{bmatrix}\right)=\begin{bmatrix}0.98\\ 0.12\end{bmatrix}$$

$$\begin{bmatrix}4\\ -2\end{bmatrix}$$

The variables that make Forward propagation and tensor shapes work

Model and assumptions

Forward propagation applies layer operations in topological order while caching the values required by the loss and backward pass.

$$Z^{(\ell)} = A^{(\ell-1)} W^{(\ell)} + \mathbf{1}(b^{(\ell)})^{\top}, \quad A^{(\ell)} = \phi(Z^{(\ell)})$$

Symbol	Meaning	Status
$A^{(\ell-1)} \in \mathbb{R}^{B \times d_{\ell-1}}$	batch activations	rows are samples
$W^{(\ell)} \in \mathbb{R}^{d_{\ell-1} \times d_{\ell}}$	weights	maps feature width
$Z^{(\ell)} \in \mathbb{R}^{B \times d_{\ell}}$	pre-activations	cached tensor

Read the numbers produced by Forward propagation and tensor shapes

Quantity	Value	Meaning
input	$32 imes 10$	batch by features
weights	$10 imes 8$	input to hidden
hidden	$32 imes 8$	valid product

What the result means

Treat shapes as part of the mathematical specification and assert them in code.

Three conditions for trusting Forward propagation and tensor shapes

Main idea

Shape annotations expose most implementation errors before optimization begins.

What to check

shape assertions and finite activation values

Limit

implicit broadcasting can produce a plausible tensor with the wrong semantics and no runtime error

An implementation checklist for Forward propagation and tensor shapes

1. fix a batch-axis convention
2. annotate every parameter shape
3. assert intermediate shapes
4. cache only backward necessities
5. test one sample and one batch

Check the result

Check shape assertions and finite activation values on training and held-out examples.

Common mistake

implicit broadcasting can produce a plausible tensor with the wrong semantics and no runtime error

Reject Forward propagation and tensor shapes under this condition

Example

implementing a two-layer network with batch-first arrays

Explain the calculation and check shape assertions and finite activation values, using a small example.

Common mistake

implicit broadcasting can produce a plausible tensor with the wrong semantics and no runtime error

What to do

Treat shapes as part of the mathematical specification and assert them in code.

Forward propagation and tensor shapes: check your understanding

Question

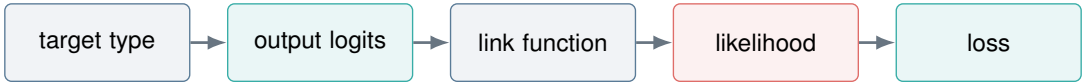
How do we make every forward-pass operation dimensionally auditable?

Answer

Shape annotations expose most implementation errors before optimization begins.

Decision. Treat shapes as part of the mathematical specification and assert them in code.

Which output transformation and loss match the random variable being predicted?



Linear outputs fit real values, sigmoid fits Bernoulli probability, and softmax fits one categorical outcome.

Question

Which output transformation and loss match the random variable being predicted?

Example

three-class image classification with imbalanced labels

Main idea

Linear outputs fit real values, sigmoid fits Bernoulli probability, and softmax fits one categorical outcome.

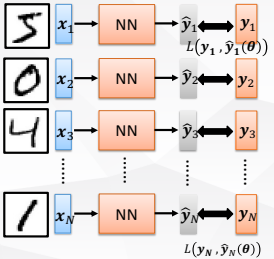
The output and target define the training loss

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

目标函数

$$\boldsymbol{\theta} = \{\boldsymbol{W}^{(1)}, \boldsymbol{b}^{(1)}, \boldsymbol{W}^{(2)}, \boldsymbol{b}^{(2)}, \cdots \boldsymbol{W}^{(L)}, \boldsymbol{b}^{(L)}\}$$

给定一组网络参数$\boldsymbol{\theta}$, 可以计算每个样本对应的损失$L(\boldsymbol{y}_i, \hat{\boldsymbol{y}}_i(\boldsymbol{\theta}))$,



目标函数：

$$J(\boldsymbol{\theta}) = \frac{1}{N}\sum_{i=1}^{N} L(\boldsymbol{y}_i, \hat{\boldsymbol{y}}_i(\boldsymbol{\theta})) + \lambda \|\boldsymbol{W}\|_F^2$$

找到使得目标函数最小的网络参数 $\boldsymbol{\theta}^*$

交叉熵

$$L(\boldsymbol{y}, \hat{\boldsymbol{y}}) = -\sum_{k=1}^{10} y_k \log(\hat{y}_k)$$

Output layers, likelihoods, and losses: model and variables

Model and assumptions

The output layer parameterizes a predictive distribution; the loss is typically its negative log-likelihood or a task-specific surrogate.

$$p_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}, \quad \mathcal{L}_{\mathrm{CE}} = -\sum_k y_k \log p_k, \quad c = \max_j z_j$$

Symbol	Meaning	Status
z_k	class logit	unbounded score
p_k	normalized probability	sums to one
c	stability shift	prevents overflow

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Output layers, likelihoods, and losses: worked example

Quantity	Value	Meaning
logits	$(2, 1, 0)$	three classes
shift	$c = 2$	stable exponents
probabilities	$(.665, .245, .090)$	normalized

What the result means

Keep a documented logits-to-loss contract and test it on extreme values.

A decision card for Output layers, likelihoods, and losses

Main idea

Linear outputs fit real values, sigmoid fits Bernoulli probability, and softmax fits one categorical outcome.

What to check

negative log-likelihood and calibration

Limit

applying softmax before a loss that already expects logits duplicates the transformation and can destabilize gradients

Reproduce the result of Output layers, likelihoods, and losses

```
def forward(X):
    Z1 = X @ W1 + b1
    A1 = np.tanh(Z1)
    Z2 = A1 @ W2 + b2
    P = sigmoid(Z2)
    return Z1, A1, Z2, P
def binary_loss(y, p):
    p = np.clip(p, 1e-9, 1-1e-9)
    return float(-np.mean(y*np.log(p) + (1-y)*np.log(1-p)))
print('Initial loss:', round(binary_loss(y, forward(X)[-1]), 4))
```

Evidence and failure check

Check negative log-likelihood and calibration with the fitted model and data split; reject the run if applying softmax before a loss that already expects logits duplicates the transformation and can destabilize gradients.

A stress test for Output layers, likelihoods, and losses

Example

three-class image classification with imbalanced labels

Explain the calculation and check negative log-likelihood and calibration, using a small example.

Common mistake

applying softmax before a loss that already expects logits duplicates the transformation and can destabilize gradients

What to do

Keep a documented logits-to-loss contract and test it on extreme values.

Output layers, likelihoods, and losses: check your understanding

Question

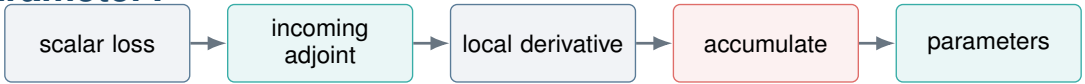
Which output transformation and loss match the random variable being predicted?

Answer

Linear outputs fit real values, sigmoid fits Bernoulli probability, and softmax fits one categorical outcome.

Decision. Keep a documented logits-to-loss contract and test it on extreme values.

How does one output loss produce derivatives for every earlier parameter?



Shared intermediate derivatives are computed once and reused, which makes reverse mode efficient for scalar losses and many parameters.

Question

How does one output loss produce derivatives for every earlier parameter?

Example

differentiating a scalar loss through a branched computation graph

Main idea

Shared intermediate derivatives are computed once and reused, which makes reverse mode efficient for scalar losses and many parameters.

Backpropagation applies the chain rule

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

链式法则

（1）若 $x \in \mathbb{R}, \boldsymbol{u} = u(x) \in \mathbb{R}^s, \boldsymbol{g} = g(\boldsymbol{u}) \in \mathbb{R}^t$，则

$$\frac{\partial \boldsymbol{g}}{\partial x} = \frac{\partial \boldsymbol{u}}{\partial x} \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{u}} \quad \in \mathbb{R}^{1 \times t}.$$

（2）若 $\boldsymbol{x} \in \mathbb{R}^p, \boldsymbol{y} = g(\boldsymbol{x}) \in \mathbb{R}^s, \boldsymbol{z} = f(\boldsymbol{y}) \in \mathbb{R}^t$，则

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}} \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{y}} \quad \in \mathbb{R}^{p \times t}.$$

（3）若 $X \in \mathbb{R}^{p \times q}$ 为矩阵，$\boldsymbol{y} = g(X) \in \mathbb{R}^s, z = f(\boldsymbol{y}) \in \mathbb{R}$，则

$$\frac{\partial z}{\partial X_{ij}} = \frac{\partial \boldsymbol{y}}{\partial X_{ij}} \frac{\partial z}{\partial \boldsymbol{y}} \quad \in \mathbb{R}.$$

Backpropagation as reverse-mode chain rule: model and variables

Model and assumptions

Backpropagation traverses the computation graph in reverse, multiplying each incoming adjoint by a cached local derivative.

$$\bar{v} = \frac{\partial \mathcal{L}}{\partial v}, \qquad \bar{u} \mathrel{+}= \bar{v} \frac{\partial v}{\partial u}$$

Symbol	Meaning	Status
v	downstream node	known incoming gradient
u	parent node	receives accumulated gradient
$\bar{v}$	adjoint	loss sensitivity

Backpropagation as reverse-mode chain rule: worked example

Quantity	Value	Meaning
forward	$v = u^2$	cache $u = 3$
incoming	$\bar{v} = 4$	loss sensitivity
backward	$\bar{u} = 4 \cdot 6 = 24$	chain rule

What the result means

Implement backward rules locally and verify branching nodes with a hand-worked graph.

Backpropagation as reverse-mode chain rule: checking the result

Main idea

Shared intermediate derivatives are computed once and reused, which makes reverse mode efficient for scalar losses and many parameters.

What to check

agreement with a numerical gradient

Limit

overwriting instead of accumulating gradients at a branch loses one dependency path

Backpropagation as reverse-mode chain rule: implementation

1. draw the computation graph
2. cache forward values
3. seed loss adjoint with one
4. apply one local rule per edge
5. sum gradients at branches

Check the result

Check agreement with a numerical gradient on training and held-out examples.

Common mistake

overwriting instead of accumulating gradients at a branch loses one dependency path

What breaks first in Backpropagation as reverse-mode chain rule

Example

differentiating a scalar loss through a branched computation graph

Explain the calculation and check agreement with a numerical gradient, using a small example.

Common mistake

overwriting instead of accumulating gradients at a branch loses one dependency path

What to do

Implement backward rules locally and verify branching nodes with a hand-worked graph.

Backpropagation as reverse-mode chain rule: check your understanding

Question

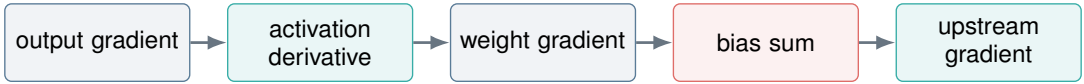
How does one output loss produce derivatives for every earlier parameter?

Answer

Shared intermediate derivatives are computed once and reused, which makes reverse mode efficient for scalar losses and many parameters.

Decision. Implement backward rules locally and verify branching nodes with a hand-worked graph.

What are the matrix forms of weight, bias, and activation gradients for a dense layer?



The weight gradient is an outer-product sum, the bias gradient is a batch sum, and the upstream gradient reverses the affine map.

Question

What are the matrix forms of weight, bias, and activation gradients for a dense layer?

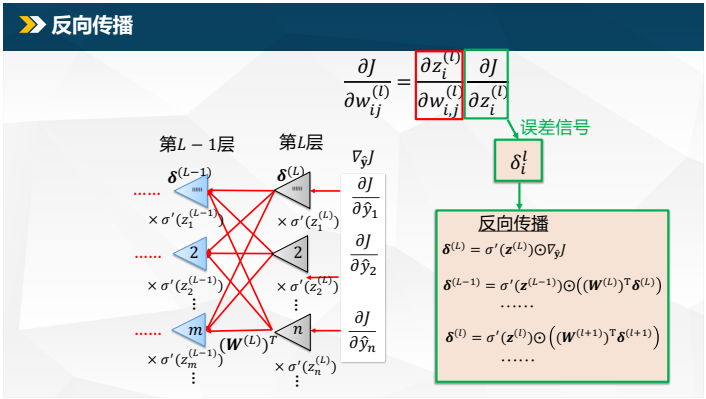
Example

deriving a batch-first dense layer backward pass

Main idea

The weight gradient is an outer-product sum, the bias gradient is a batch sum, and the upstream gradient reverses the affine map.

A weight gradient combines input and error



Vectorized layer gradients: model and variables

Model and assumptions

For a batch-first dense layer, matrix calculus expresses all sample contributions as products and reductions.

$$\nabla_W \mathcal{L} = A^\top \Delta, \quad \nabla_b \mathcal{L} = \mathbf{1}^\top \Delta, \quad \nabla_A \mathcal{L} = \Delta W^\top$$

Symbol	Meaning	Status
A	layer input	$B \times d_{in}$
Δ	pre-activation gradient	$B \times d_{out}$
∇_W	weight gradient	$d_{in} \times d_{out}$

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Vectorized layer gradients: worked example

Quantity	Value	Meaning
A	32×10	batch input
Δ	32×8	local gradient
$A^\top \Delta$	10×8	matches weights

What the result means

Use one reduction convention across the loss and every gradient.

Accept or reject Vectorized layer gradients

Main idea

The weight gradient is an outer-product sum, the bias gradient is a batch sum, and the upstream gradient reverses the affine map.

What to check

gradient shape, norm, and vectorized-loop agreement

Limit

averaging the loss but summing one parameter gradient changes the effective learning rate by batch size

Vectorized layer gradients: implementation

```
def gradients(X, y):
    Z1, A1, Z2, P = forward(X)
    dZ2 = (P - y) / len(X)
    dW2 = A1.T @ dZ2
    db2 = dZ2.sum(axis=0, keepdims=True)
    dZ1 = (dZ2 @ W2.T) * (1.0 - A1**2)
    dW1 = X.T @ dZ1
    db1 = dZ1.sum(axis=0, keepdims=True)
    return dW1, db1, dW2, db2
for _ in range(800):
    dW1, db1, dW2, db2 = gradients(X, y)
    W1 -= 0.8*dW1; b1 -= 0.8*db1; W2 -= 0.8*dW2; b2 -= 0.8*db2
print('Trained loss/predictions:', round(binary_loss(y, forward(X)[-1]), 4), (forward(X)[-1] > .5).astype(int).ravel
    ())
```

Evidence and failure check

Check gradient shape, norm, and vectorized-loop agreement with the fitted model and data split; reject the run if averaging the loss but summing one parameter gradient changes the effective learning rate by batch size.

The hidden assumption in Vectorized layer gradients

Example

deriving a batch-first dense layer backward pass

Explain the calculation and check gradient shape, norm, and vectorized-loop agreement, using a small example.

Common mistake

averaging the loss but summing one parameter gradient changes the effective learning rate by batch size

What to do

Use one reduction convention across the loss and every gradient.

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Vectorized layer gradients: check your understanding

Question

What are the matrix forms of weight, bias, and activation gradients for a dense layer?

Answer

The weight gradient is an outer-product sum, the bias gradient is a batch sum, and the upstream gradient reverses the affine map.

Decision. Use one reduction convention across the loss and every gradient.

Initialization affects signal variance across layers

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

每层输出和梯度的方差

- 略去非线性变换部分，全连接每层输出$\boldsymbol{o}$和输入$\boldsymbol{x}$之间的关系为

$$o_i = \sum_{j=1}^{N_{in}} w_{i,j}\, x_j$$

N_{in}: 神经元的输入维度

- 将每层的输出和梯度看成随机变量
- 假设所有$w_{i,j}$独立同分布，x_j独立同分布，且$w_{i,j}$和x_j独立

$$\mathbb{E}[w_{i,j}] = 0,\ \mathrm{Var}[w_{i,j}] = \sigma^2, \qquad \mathbb{E}[x_j] = 0,\ \mathrm{Var}[x_j] = \gamma^2$$

- 则输出的期望和方差为：

$$\mathbb{E}[o_i] = \mathbb{E}[\textstyle\sum_{j=1}^{N_{\mathrm{in}}} w_{i,j}\, x_j]$$

$$= \sum_{j=1}^{N_{\mathrm{in}}} \mathbb{E}[w_{i,j} x_j] = \sum_{j=1}^{N_{\mathrm{in}}} \mathbb{E}[w_{i,j}]\mathbb{E}[x_j] = 0$$

He initialization sets variance using the number of inputs

- For a ReLU layer, a common choice is $\mathrm{Var}(w) = 2/\mathrm{fan_in}$.
- With 100 incoming connections, variance is 0.02 and standard deviation is about 0.141.
- Sample independent weights to break symmetry.
- This is an initial scale choice; inspect activations and gradients after initialization.



Initialization and signal variance: model and variables

Model and assumptions

Variance-scaled initialization chooses weight variance from fan-in or fan-out so signals neither explode nor vanish at the start.

$$\mathrm{Var}(W_{ij}) \approx \frac{1}{d_{in}} \text{ (Xavier)}, \qquad \mathrm{Var}(W_{ij}) \approx \frac{2}{d_{in}} \text{ (He)}$$

Symbol	Meaning	Status
d_{in}	fan-in	number of inputs per unit
Xavier	variance balance	tanh or linear-like
He	ReLU-aware scale	rectified networks

Initialization and signal variance: worked example

Quantity	Value	Meaning
fan-in	256	ReLU layer
He variance	$2/256$	target
standard deviation	0.088	sampling scale

What the result means

Use variance-scaled initialization and verify the realized statistics before full training.

Separate a good fit from a good decision in Initialization and signal variance

Main idea

Xavier scaling suits roughly symmetric activations; He scaling compensates for ReLU discarding about half the signal.

What to check

layerwise activation and gradient variance

Limit

large arbitrary weights saturate smooth activations or amplify signals exponentially across depth

Initialization and signal variance: implementation

1. choose scale from activation family
2. initialize biases conservatively
3. run one forward batch
4. plot layerwise variances
5. run one backward batch

Check the result

Check layerwise activation and gradient variance on training and held-out examples.

Common mistake

large arbitrary weights saturate smooth activations or amplify signals exponentially across depth

Do not trust Initialization and signal variance before this check

Example

initializing a ten-layer ReLU network

Explain the calculation and check layerwise activation and gradient variance, using a small example.

Common mistake

large arbitrary weights saturate smooth activations or amplify signals exponentially across depth

What to do

Use variance-scaled initialization and verify the realized statistics before full training.

Initialization and signal variance: check your understanding

Question

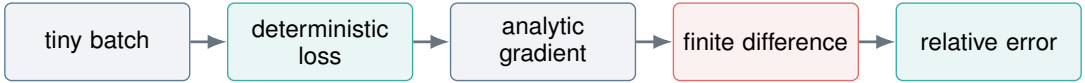
How should initial weights preserve activation and gradient scale across depth?

Answer

Xavier scaling suits roughly symmetric activations; He scaling compensates for ReLU discarding about half the signal.

Decision. Use variance-scaled initialization and verify the realized statistics before full training.

How do we distinguish a calculus bug from an optimization or data problem?



A passing local gradient check validates derivative code, but not the data pipeline, optimization schedule, or generalization.

Question

How do we distinguish a calculus bug from an optimization or data problem?

Example

debugging a custom two-layer binary classifier

Main idea

A passing local gradient check validates derivative code, but not the data pipeline, optimization schedule, or generalization.

Check a gradient with a central difference

- $g_j^{\mathrm{num}} = \dfrac{L(\theta + \epsilon e_j) - L(\theta - \epsilon e_j)}{2\epsilon}$.
- Compare with backpropagation using relative as well as absolute error.
- Use a tiny deterministic example in double precision; disable dropout.
- Try several ϵ values and avoid checking exactly at ReLU kinks.

Gradient checking and debugging protocol: model and variables

Model and assumptions

Gradient checking compares analytic derivatives with centered finite differences on a tiny deterministic problem.

$$g_j^{num} = \frac{\mathcal{L}(\theta_j + \varepsilon) - \mathcal{L}(\theta_j - \varepsilon)}{2\varepsilon}, \quad \delta = \frac{\|g^{ana} - g^{num}\|}{\|g^{ana}\| + \|g^{num}\|}$$

Symbol	Meaning	Status
ε	finite-difference step	small but not underflowing
g^{ana}	backprop gradient	implementation under test
δ	relative discrepancy	scale-aware diagnostic

Gradient checking and debugging protocol: worked example

Quantity	Value	Meaning
g^{ana}	0.125001	backprop
g^{num}	0.124999	finite difference
δ	8×10^{-6}	acceptable for this scale

What the result means

Require both a stable finite-difference check and successful tiny-batch overfitting before large experiments.

Gradient checking and debugging protocol: checking the result

Main idea

A passing local gradient check validates derivative code, but not the data pipeline, optimization schedule, or generalization.

What to check

relative gradient error and tiny-batch loss

Limit

finite differences with an extreme step can fail from truncation or floating-point cancellation even when calculus is correct

Gradient checking and debugging protocol: implementation

```
theta = W1.flat[0]
eps = 1e-5
W1.flat[0] = theta + eps; lp = binary_loss(y, forward(X)[-1])
W1.flat[0] = theta - eps; lm = binary_loss(y, forward(X)[-1])
W1.flat[0] = theta
g_num = (lp-lm)/(2*eps)
g_ana = gradients(X, y)[0].flat[0]
rel_error = abs(g_num-g_ana)/(abs(g_num)+abs(g_ana)+1e-12)
print('Gradient check:', g_num, g_ana, 'relative error', rel_error)
```

Evidence and failure check

Check relative gradient error and tiny-batch loss with the fitted model and data split; reject the run if finite differences with an extreme step can fail from truncation or floating-point cancellation even when calculus is correct.

Gradient checking and debugging protocol: limitations

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Example

debugging a custom two-layer binary classifier

Explain the calculation and check relative gradient error and tiny-batch loss, using a small example.

Common mistake

finite differences with an extreme step can fail from truncation or floating-point cancellation even when calculus is correct

What to do

Require both a stable finite-difference check and successful tiny-batch overfitting before large experiments.

Gradient checking and debugging protocol: check your understanding

Question

How do we distinguish a calculus bug from an optimization or data problem?

Answer

A passing local gradient check validates derivative code, but not the data pipeline, optimization schedule, or generalization.

Decision. Require both a stable finite-difference check and successful tiny-batch overfitting before large experiments.

Walk XOR through the hidden representation

Decision

Plot the four inputs, compute two hidden activations, and show how the hidden plane makes the classes linearly separable.

Evidence

Keep the weights visible so geometry and algebra refer to the same network.

Stop condition

Do not claim that depth helps until the intermediate representation is actually inspected.

Trace one scalar loss backward

Decision

Start at the loss, write each local derivative beside its forward operation, and multiply only along valid graph paths.

Evidence

Check signs and tensor shapes before substituting numbers.

Stop condition

Stop when a gradient reaches a parameter with the wrong shape or bypasses a required branch.

Use a shape ledger for every layer

Decision

Record batch, feature, hidden, and output dimensions for activations, weights, biases, and gradients.

Evidence

Make broadcasting explicit instead of relying on silent library behavior.

Stop condition

Treat a numerically plausible result as wrong when its axes do not match the model definition.

Gradient checking is a local debugging tool

Decision

Compare analytic and finite-difference gradients on a tiny deterministic network in double precision.

Evidence

Use relative error and inspect one parameter block at a time.

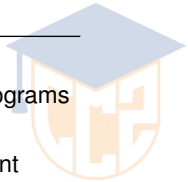
Stop condition

Do not run gradient checking as a training method or accept disagreement caused by nondifferentiable points without analysis.

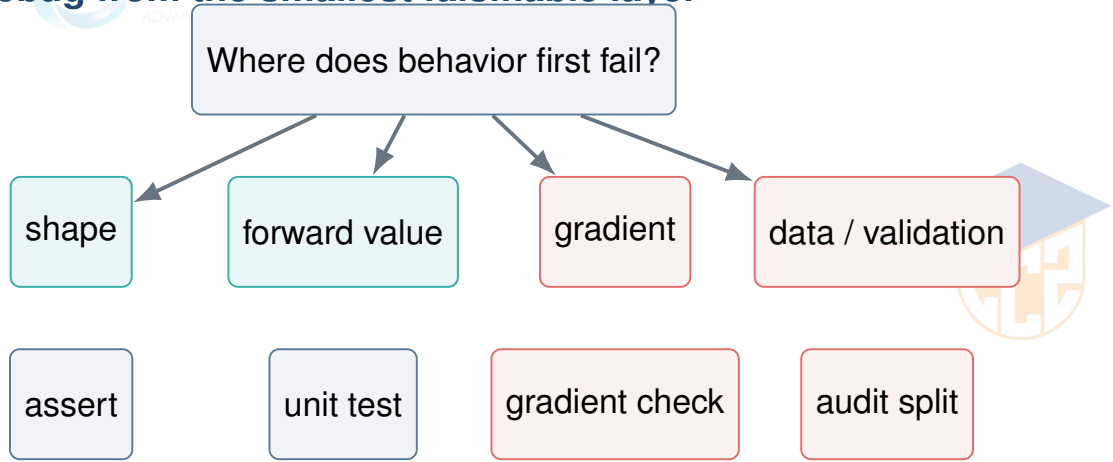
先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Compare network components by their mathematical role

Component	Primary role	First diagnostic
affine layer	mix coordinates	matrix shapes
activation	create nonlinearity	value and derivative histograms
output plus loss	model target distribution	likelihood and calibration
backpropagation	compute parameter sensitivities	finite-difference agreement



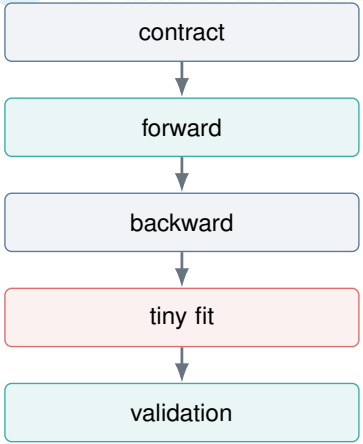
Debug from the smallest falsifiable layer



Start from the deliverable

Start with the observable symptom, then test shapes, forward values, local derivatives, parameter updates, and only then generalization.

A reliable neural-network implementation workflow



1. fix the data and target contract
2. annotate and assert every tensor shape
3. test stable forward values
4. gradient-check a tiny deterministic graph
5. overfit one tiny batch
6. only then tune on dependency-aware validation

Each stage should fail loudly before the next stage begins.

Use relative error and learning behavior together

$$\delta_g = \frac{\|g^{ana} - g^{num}\|_2}{\|g^{ana}\|_2 + \|g^{num}\|_2}, \qquad \Delta L = L_0 - L_T$$

Observation	Likely issue	Next action
shape mismatch	interface contract	fix axes and broadcasting
gradient check fails	backward rule	localize parameter subset
tiny batch will not fit	optimization or model	inspect activations and updates

Stress-test the implementation

先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

mathematical contract | numerical behavior | generalization

- test batch size one and multiple batches
- test extreme logits and stable losses
- sweep finite-difference epsilon
- inspect activation and gradient norms
- repeat initialization seeds
- compare tiny-batch and held-out behavior



先进计算与数字工程研究中心
ADVANCED COMPUTING AND DIGITAL TECHNOLOGY RESEARCH CENTER

Six takeaways to retain

1. a perceptron is one hyperplane
2. nonlinear hidden layers learn representations
3. outputs and losses encode a distribution
4. backprop is reverse-mode chain rule
5. matrix gradients must match parameter shapes
6. gradient checks precede scale

Exit ticket: diagnose a network that does not learn

Prompt

Name the first check you would run, the tensor or derivative it tests, the expected evidence, and the next action for each possible result.

Case

A two-layer classifier stays near chance. The loss is finite, but hidden activations are almost constant and the first-layer gradient norm is 10^{-9}.